

1. 위의 $m = 1 \sim p$ 까지 나열된 식을 참고하여 R matrix (square matrix)를 구하고 이를 이용하여 Yule-Walker equation을 나타내세요. (보고서에 설계한 행렬식을 첨부하세요.)

[Hint]

AR model의 coefficient는 Yule-Walker equation ($\mathbf{R} \cdot \mathbf{a} = \mathbf{r}$)에서 inverse matrix를 이용하여 구할 수 있습니다. 특히, $\mathbf{R}$ 이 square matrix일 경우 다음의 식으로 구할 수 있습니다.

$$\mathbf{a} = \mathbf{R}^{-1} \cdot \mathbf{r} \quad (1)$$

$$\textcircled{\bullet} r_{xx}[m] = -\sum_{k=1}^p a[k] \cdot r_{xx}[m-k]$$

$$m=0, \quad r_{xx}[0] = -a[1]r_{xx}[-1] - a[2]r_{xx}[-2] - \dots - a[p]r_{xx}[-p] + S_u$$

$$m=1, \quad r_{xx}[1] = -a[1]r_{xx}[0] - a[2]r_{xx}[-1] - \dots - a[p]r_{xx}[-p+1]$$

$\vdots$

$$m=p, \quad r_{xx}[p] = -a[1]r_{xx}[p-1] - a[2]r_{xx}[p-2] - \dots - a[p]r_{xx}[0]$$

$$\begin{aligned} \text{---} \quad \mathbf{R} \cdot \mathbf{a} &= \begin{bmatrix} r_{xx}[0] & r_{xx}[-1] & r_{xx}[-2] & \dots & r_{xx}[-p+1] \\ r_{xx}[1] & r_{xx}[0] & r_{xx}[-1] & \dots & r_{xx}[-p+2] \\ r_{xx}[2] & r_{xx}[1] & r_{xx}[0] & \dots & r_{xx}[-p+3] \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ r_{xx}[p-1] & r_{xx}[p-2] & r_{xx}[p-3] & \dots & r_{xx}[0] \end{bmatrix} \begin{bmatrix} -a[1] \\ -a[2] \\ -a[3] \\ \vdots \\ -a[p] \end{bmatrix} = \begin{bmatrix} r_{xx}[1] \\ r_{xx}[2] \\ r_{xx}[3] \\ \vdots \\ r_{xx}[p] \end{bmatrix} = \mathbf{r} \end{aligned}$$

$\Rightarrow$ 위를 통해 $\mathbf{a} = \mathbf{R}^{-1} \cdot \mathbf{r}$ 이용 하여 $\mathbf{a}$ 구할 수 있음.

2. $m = 1 \sim 2p$ 까지 $r_{xx}[m]$ 에 대한 식을 나열하고 R' matrix (non-square matrix)를 구하세요

요. (보고서에 나열된 식과 설계한 행렬식을 첨부하세요.)

[Hint]

AR model의 coefficient는 R 이 square matrix가 아니라면 문제 1의 Hint의 수식 (1)을 통해 구할 수 없습니다. Non-square matrix를 R' 이라고 할 때, 아래 수식(2)와 같이 Least Squares를 활용하여 coefficient를 구할 수 있습니다.

$$a = (R'^T R')^{-1} R'^T \cdot r \dots (2)$$

$$\textcircled{\bullet} r_{xx}[m] = -\sum_{k=1}^p a[k] \cdot r_{xx}[m-k]$$

$$m=0, \quad r_{xx}[0] = -a[1]r_{xx}[-1] - a[2]r_{xx}[-2] - \dots - a[p]r_{xx}[-p] + S_u$$

$$m=1, \quad r_{xx}[1] = -a[1]r_{xx}[0] - a[2]r_{xx}[-1] - \dots - a[p]r_{xx}[-p+1]$$

$\vdots$

$$m=p, \quad r_{xx}[p] = -a[1]r_{xx}[p-1] - a[2]r_{xx}[p-2] - \dots - a[p]r_{xx}[0]$$

$\vdots$

$$m=2p, \quad r_{xx}[2p] = -a[1]r_{xx}[2p-1] - a[2]r_{xx}[2p-2] - \dots - a[p]r_{xx}[p]$$



$$R' \cdot a = \begin{bmatrix} r_{xx}[0] & r_{xx}[-1] & r_{xx}[-2] & \dots & r_{xx}[-p+1] \\ r_{xx}[1] & r_{xx}[0] & r_{xx}[-1] & \dots & r_{xx}[-p+2] \\ r_{xx}[2] & r_{xx}[1] & r_{xx}[0] & \dots & r_{xx}[-p+3] \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ r_{xx}[p-1] & r_{xx}[p-2] & r_{xx}[p-3] & \dots & r_{xx}[0] \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ r_{xx}[2p-1] & r_{xx}[2p-2] & r_{xx}[2p-3] & \dots & r_{xx}[p] \end{bmatrix} \begin{bmatrix} -a[1] \\ -a[2] \\ -a[3] \\ \vdots \\ -a[p] \end{bmatrix} = \begin{bmatrix} r_{xx}[1] \\ r_{xx}[2] \\ r_{xx}[3] \\ \vdots \\ r_{xx}[p] \\ \vdots \\ r_{xx}[2p] \end{bmatrix} = r$$

$\Rightarrow$ 위를 통해 Least Square Inverse

$$a = (R'^T R')^{-1} R'^T \cdot r \quad \text{이름 하여 } a \text{ 구할 수 있음.}$$

3. 문제 1의 [Hint]의 수식(1)을 이용하여 각 입력신호에 대한 AR(3), AR(7), AR(13)의 coefficient $a[n]$ 을 구하세요. 3, 7, 13은 pole의 개수를 의미합니다. ($a[0] = 1$)

[Hint]

$r_{xx}[m]$ 은 다음과 같이 구할 수 있으며, $r_{xx}[-k] = r_{xx}^*[k]$ 의 성질이 있습니다.
(N 은 주어진 신호 $x[n]$ 의 길이입니다.)

$$r_{xx}[m] = E\{x[n]x^*[n+m]\} = \frac{1}{N-|m|} \sum_{n=0}^{N-|m|-1} x[n+|m|]x^*[n]$$

Autocorrelation $r_{xx}[m]$ 을 구할 때 제공된 함수를 사용하세요.

※ 제공된 함수 참고 : `r_xx()`

Inverse matrix를 구할 때 내장함수를 이용하세요.

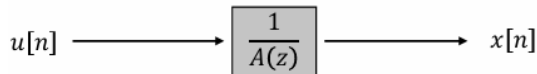
※ 내장함수 참고 : `inv()`

```
AR(3) coefficients:
1.0000
-1.3388
-0.2061
0.5739

AR(7) coefficients:
1.0000
-0.9394
-0.3620
-0.1678
0.1885
0.3195
0.1918
-0.1857

AR(13) coefficients:
1.0000
-0.8388
-0.3635
-0.2413
0.0710
0.2253
0.1801
-0.0720
0.0401
0.0892
0.0645
-0.0192
0.0377
-0.1536
```

[이론 설명]



위와 같은 all pole System에서 System에 해당하는 $\frac{1}{A(z)}$ 를 구하려면 아래와 같이 수식을 전개할 수 있다.

- 1) Error를 정의하고 Error의 Energy가 최소가 되는 $a[k]$ 가 system의 coefficient라고 가정

$$\min_{a[k]} \left\{ \sum_{n=C_1}^{C_2} |u[n]|^2 \right\} \Rightarrow \min_{a[k]} \left\| \sum_{n=C_1}^{C_2} \left[x[n] + \sum_{k=1}^p a[k]x[n-k] \right] \right\|^2 = 0$$

- 2) Error의 Energy가 최소가 되는 $a[k]$ 는 Orthogonality Principle에 의해 쉽게 결정할 수 있음

즉, $E\{u[n]x^*[n-m]\}$ 이 0이 되는 자기자신과의 correlation을 제외한 구간에서 0이면 됨.

$$x[n] = -\sum_{k=1}^p a[k]x[n-k] + u[n]$$

$$x[n]x^*[n-m] = -\sum_{k=1}^p a[k]x[n-k]x^*[n-m] + u[n]x^*[n-m]$$

$$E\{x[n]x^*[n-m]\} = -\sum_{k=1}^p a[k]E\{x[n-k]x^*[n-m]\} + E\{u[n]x^*[n-m]\}$$

$$\gamma_{xx}[m] = -\sum_{k=1}^p a[k]\gamma_{xx}[m-k] + \gamma_{ux}[m]$$

$$\gamma_{ux}[m] = E\{u[n](-\sum_{k=1}^p a[k]x^*[n-m-k] + u^*[n-m])\} = -\sum_{k=1}^p a[k]E\{u[n]x^*[n-m-k]\} + E\{u[n]u^*[n-m]\}$$

$$\gamma_{ux}[m] = \begin{cases} \sigma_u, & m = 0 \\ 0, & m \neq 0 \end{cases}$$

$$\gamma_{xx}[m] = -\sum_{k=1}^p a[k]\gamma_{xx}[m-k] + \sigma_u\delta[m]$$

위 일련의 과정을 통해 $\gamma_{ux}[m]$ 은 lag = 0 일 자기자신과의 correlation 을 제외하면 correlation 결과가 모두 0 임을 알 수 있고, $\gamma_{xx}[m] = -\sum_{k=1}^p a[k]\gamma_{xx}[m-k] + \sigma_u\delta[m]$ 를 Matrix 형태로 풀어 쓰면 문제 1 에서 구한 행렬식이 된다.

문제 1에서 언급한 $a = R^{-1} \cdot r$ 을 사용하여 a 를 구하는 것을 문제 3에서 구현한다.

이를 위해 r_{xx} 가 필요하다. 따라서, [Hint]에 적혀 있는 auto correlation 정의에 따라, r_{xx} 를 생성해 준다. 이번 실습에서는 skeleton code로 r_{xx} 함수가 주어졌다.

따라서, 문제 1에서 구한 식에 따라 r_{xx} 함수를 이용하여 r_{xx} vector와 Toeplitz Matrix를 구하고

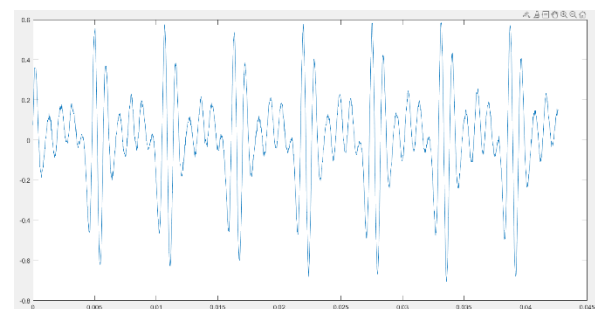
Toeplitz Matrix의 inverse를 구하여 최종적으로 coefficient a를 구한다.

```
r(1:13) as row vector:
5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02  2.5421e-02  1.9415e-02  1.3166e-02  6.7961e-03  4.2621e-04

Toeplitz Matrix R (13 x 13):
      1      2      3      4      5      6      7      8      9     10     11     12     13
1  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02  2.5421e-02  1.9415e-02  1.3166e-02  6.7961e-03  4.2621e-04
2  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02  2.5421e-02  1.9415e-02  1.3166e-02  6.7961e-03
3  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02  2.5421e-02  1.9415e-02  1.3166e-02
4  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02  2.5421e-02  1.9415e-02
5  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02  2.5421e-02
6  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02  3.1054e-02
7  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02  3.6188e-02
8  3.1054e-02  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02  4.0739e-02
9  2.5421e-02  3.1054e-02  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02  4.4608e-02
10 1.9415e-02  2.5421e-02  3.1054e-02  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02  4.7704e-02
11 1.3166e-02  1.9415e-02  2.5421e-02  3.1054e-02  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02  4.9954e-02
12 6.7961e-03  1.3166e-02  1.9415e-02  2.5421e-02  3.1054e-02  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02  5.1333e-02
13 4.2621e-04  6.7961e-03  1.3166e-02  1.9415e-02  2.5421e-02  3.1054e-02  3.6188e-02  4.0739e-02  4.4608e-02  4.7704e-02  4.9954e-02  5.1333e-02  5.1852e-02
```

[Hint]에 언급하였듯이 auto correlation은 $r_{xx}[-k] = r_{xx}^*[k]$ (Hermitian 대칭)이다.

추가로 이번 실습의 $x[n]$ 의 파형을 그려보면, console창에 “imaginary신호가 포함되어 이를 무시합니다.” 와 같은 예러가 뜨지 않고, 아래의 파형이 제대로 그려지므로, $x[n]$ 은 실수 신호임을 알 수 있다. 실수 신호의 경우에는 imaginary영역이 없기 때문에 $r_{xx}[-k] = r_{xx}[k]$ 와 같이 작성할 수 있다.



4. 문제 3에서 구한 계수들을 이용하여 PSD(Power Spectral Density)의 그래프를 log scale 로 그리세요. ($a[0] = 1$)

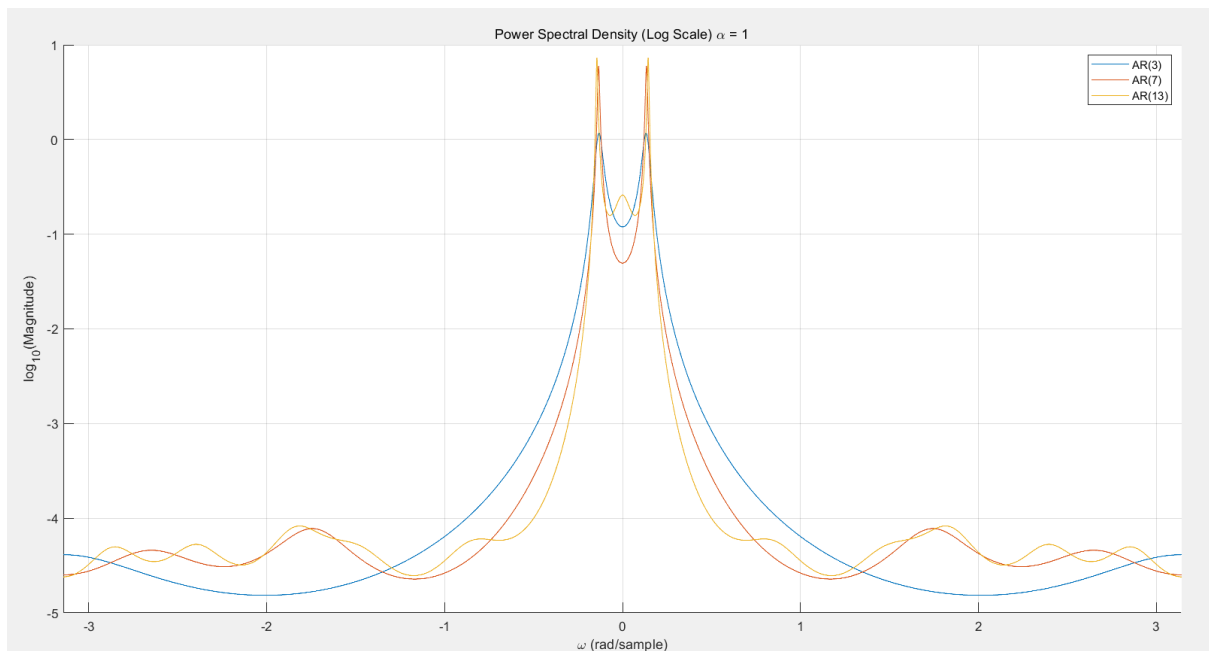
[Hint]

PSD는 다음의 식으로 구할 수 있습니다.

$$P_{AR}(\omega) = \frac{\beta^2}{|A(\omega)|^2}, \quad A(\omega) = \sum_{k=0}^p a[k]e^{-j\omega k}$$

여기서 신호의 variance β^2 은 다음과 같이 가정합니다.

$$\beta^2 = 1 \times 10^{-4}$$



log10을 이용하여 plot한 그림

[이론 설명]

coefficient a 를 구한 이후에는 $A(z) = a[0] + a[1]z^{-1} + a[2]z^{-2} + \dots + a[p]z^{-p}$ 이므로 이 식을 이용하여 $A(z)$ 를 구한다. 이번 실습에서는 [Hint]를 참고하여 $A(\omega)$ 를 구한다.

더불어, [Hint]에 주어진 식을 이용하여 PSD를 구한다.

[그래프 분석]

AR 차수가 증가할수록 주파수 파형이 더 상세하게 그려지며, 중심 주파수 부근에서 더욱 선명한 peak가 나타났다.

AR(3): peak가 부드러우며 pole의 개수가 적어 파형의 표현이 제한적이고, detail이 부족하다.

AR(7): 더 선명한 peak가 나타나며, pole의 개수에 따라 극대점이 보이며, 파형의 구조가 더 잘 표

현된다.

AR(13): $A(z)$ 의 차수가 커져, AR(7)과 같이 선명한 peak가 나타나고, pole의 개수가 많아져 더욱 상세하게 파형을 나타낸다.

즉, pole이 많아질수록 더욱 상세하게 파형을 표현할 수 있고, peak와 극대점들이 더 많이 보인다.

하지만, pole을 여러 개로 설정한 model은 data의 noise까지 추정하려는 경향이 있어, 이는 model의 일반화 성능을 떨어뜨릴 수 있다고 한다.

따라서 모델 차수는 높다고 무조건 좋은 것이 아니며, 적절한 차수를 선택하는 것이 중요하다.

5. 문제 2의 [Hint]의 수식(2)를 참고하여, $1 \leq m \leq 2p$ 범위의 Yule-Walker equation 사용하여 AR(13)의 coefficient $a[n]$ 을 구하세요.

($a[0] = 1$), ($r_{xx}[1], r_{xx}[2], \dots, r_{xx}[2p], p = 13$)

[이론 설명]

$m = 0$ 부터 $m = 2p$ 까지 확장하여, lag(= m)을 더 많이 사용하였을 경우의 coefficient $a[n]$ 을 구한다.

문제2에서 다루었듯이 Non-Square Matrix의 Inverse는 Least Square 방식을 이용하여 구할 수 있다.

수식은 다음과 같다.

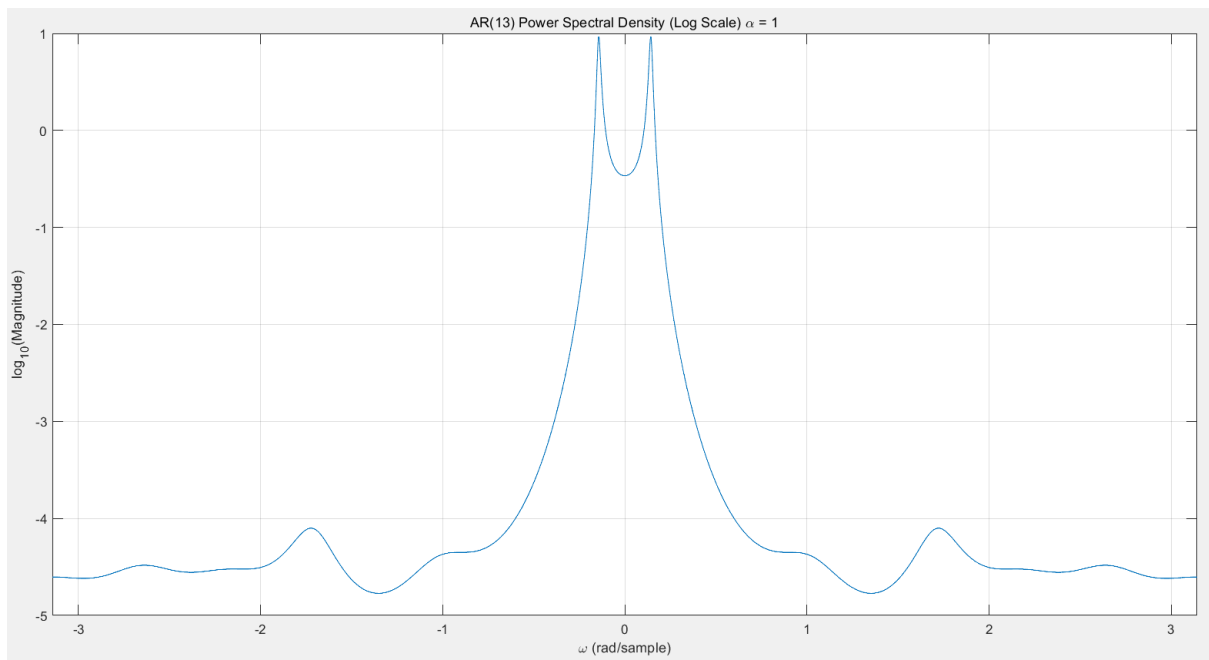
Find coefficient $\mathbf{a}$ using Least Square Inverse:

$$\mathbf{a} = (\mathbf{R}^T \mathbf{R})^{-1} \mathbf{R}^T \cdot \mathbf{r}$$

$$\mathbf{R}' \cdot \mathbf{a} = \begin{bmatrix} r_{xx}[0] & r_{xx}[-1] & \cdots & r_{xx}[-p+1] \\ r_{xx}[1] & r_{xx}[0] & \cdots & r_{xx}[-p+2] \\ \vdots & \vdots & \ddots & \vdots \\ r_{xx}[2p] & r_{xx}[2p-1] & \cdots & r_{xx}[p] \end{bmatrix} \cdot \begin{bmatrix} -a[1] \\ -a[2] \\ \vdots \\ -a[p] \end{bmatrix} = \begin{bmatrix} r_{xx}[1] \\ r_{xx}[2] \\ \vdots \\ r_{xx}[2p] \end{bmatrix} = \mathbf{r}$$

```
AR(13) coefficients (Use Least Square solution):
1.0000
-1.1396
-0.2999
-0.0376
0.2860
0.2957
0.0516
-0.1583
0.0818
0.1139
-0.0769
-0.1483
-0.0327
0.0814
```

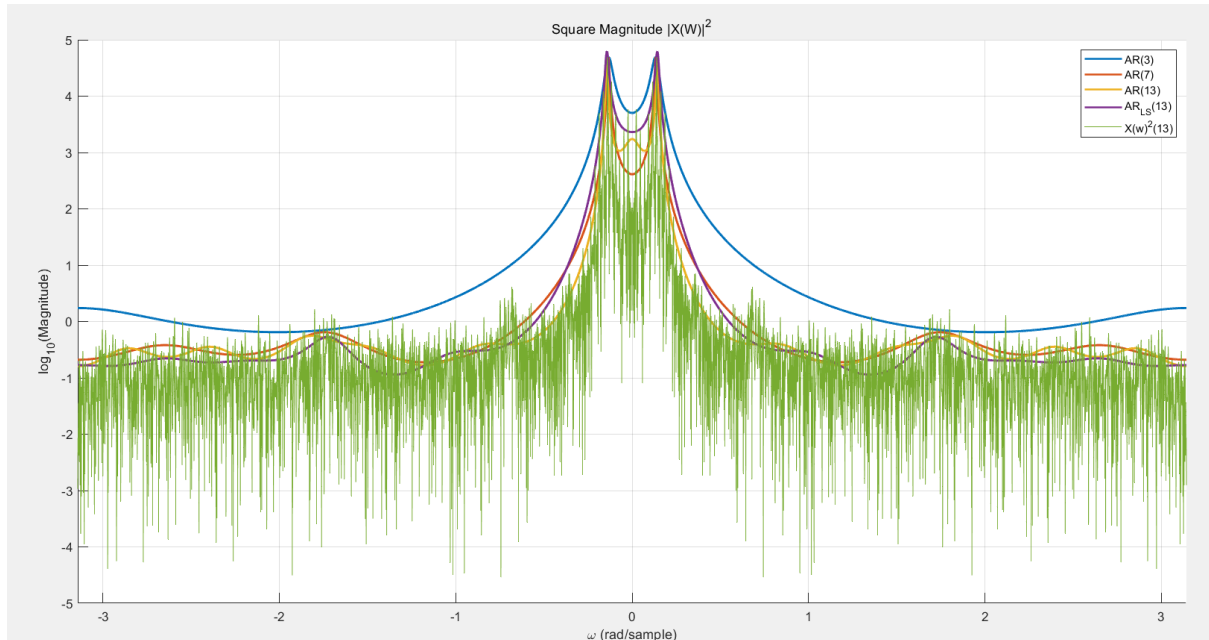
6. 위의 문제 5에서 구한 계수들을 사용하여 PSD(Power Spectral Density) 그래프를 log scale로 그리세요.



[이론 설명]

Lag를 길게 잡으면 더 많은 $r_{xx}[m]$ 을 사용한다. 이는 과거 신호의 더 긴 시간 범위를 고려하게 되어, 더 정확하게 system의 구조를 반영할 수 있다. 더불어, 실제 신호 길이에 비하여 pole의 개수는 상대적으로 적기에 lag를 늘려 더 상세한 정보를 얻는다.

7. 주어진 음성 신호의 power spectrum을 대체하여, magnitude의 제곱 그래프 ($|X(\omega)|^2$)를 사용합니다. $|X(\omega)|^2$ 을 log scale로 그린 후, 각 modeling의 PSD 그래프와 비교하세요. (AR modeling의 특징을 파악한 후 결과 그래프를 바탕으로 토의 과정을 자세히 작성하세요.)



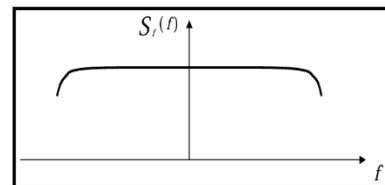
[각각의 plot 비교]

초록색	주어진 음성 신호의 magnitude의 제곱 ($ X(\omega) ^2$)
파란색	Pole을 3개로 설정하고 Direct Inverse를 사용하였을 때, 추정된 system으로부터 나온 PSD
주황색	Pole을 7개로 설정하고 Direct Inverse를 사용하였을 때, 추정된 system으로부터 나온 PSD
노란색	Pole을 13개로 설정하고 Direct Inverse를 사용하였을 때, 추정된 system으로부터 나온 PSD
보라색	Pole을 13개로 설정하고 Least Square Inverse를 사용하였을 때, 추정된 system으로부터 나온 PSD

[결과 분석]

$$X(z) = \frac{1}{A(z)} U(z), \quad \text{with } A(z) = a[0] + a[1]z^{-1} + \dots + a[p]z^{-p} \text{ 이다.}$$

AR 모델링의 경우 input 신호를 White Noise로 가정하고 모델링을 진행하기 때문에, input의 주파수 영역에서의 응답은 Flat하다고 볼 수 있다.

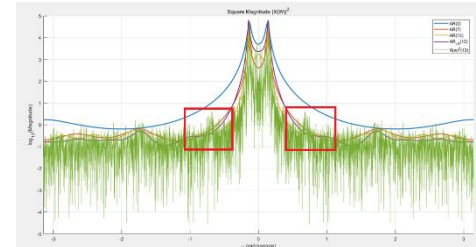


$P_{AR}(\omega) = \frac{\beta^2}{|A(\omega)|^2}$, where $A(\omega) = \sum_{k=0}^p a[k]e^{-j\omega k}$ 에서 β^2 은 White Noise의 분산으로 평균이 0이고 이 경우, Variance = $E[u^2[n]]$ = Power가 성립한다.

결국, $P_{AR}(\omega) \approx \left| \frac{1}{A(z)} U(z) \right|^2 \approx |X(\omega)|^2$ 이라고 볼 수 있는 것이다. 따라서, α 값을 잘 설정하여, $P_{AR}(\omega)$ 의 최대 크기를 $|X(\omega)|^2$ 와 동일하게 맞추고, plot을 서로 비교하면, 추정된 system $\frac{1}{A(z)}$ 가 얼마나 잘 추정하였는지를 측정할 수 있다. α 는 $\frac{\max(|X(\omega)|^2)}{\max(\text{PSD}(\omega))}$ 와 같이 설정하여 최대 Power를 동일하게 맞추었다.

파란색 plot의 경우 pole의 개수를 3개로 매우 적게 설정하여 $A(z)$ 의 차수가 적다. 따라서, System의 세부적인 요소까지 자세하게 추정하지는 못한다. 따라서, AR(3)을 통해 구한 PSD는 대략적인 분포만 부드러운 곡선의 형태로 나타나고, detail은 부족한 모습을 보인다.

주황색 plot의 경우 pole의 개수를 7개로 설정하여 $A(z)$ 를 구하였다. 따라서, AR(3)에 비하여 System의 세부적인 요소를 추정하였다. 하지만, $|X(w)|^2$ 와 비교하였을 때, 오른쪽 그림의 빨간 네모박스의 부분은 잘 반영하지 못하는 모습을 보인다. 따라서, AR(3)을 통해 구한 PSD보다는 구체적인 요소를 추정하지만, 약간의 detail을 놓치는 모습을 보인다.



노란색 plot의 경우 pole의 개수를 13개로 설정하여 $A(z)$ 를 구하였다. 이 경우 AR(3)와 AR(7)에 비하여 $A(z)$ 의 차수가 더 높으므로, 더 세밀하게 system을 추정할 수 있다. Plot된 파형을 보게 되면, 노란색 파형의 경우 주황색 파형에 비해서 한 층 더 $|X(w)|^2$ 의 파형을 잘 따라가는 것을 확인할 수 있다.

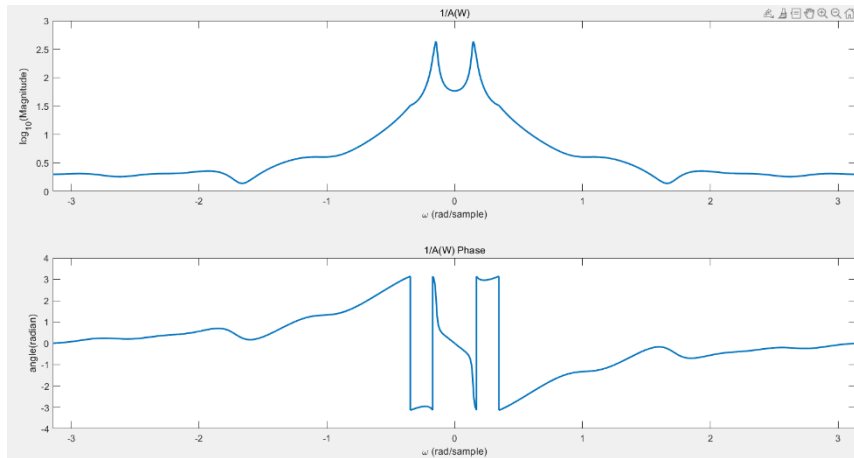
보라색 plot의 경우 pole의 개수는 13개로 노란색 plot과 동일하다. 하지만, 차이점은 lag (= m)을 더 많이 사용하였다는 것이다. lag(= m)을 더 많이 사용할 경우 $x[n]$ 과 $x[n]$ 이 delay된 신호들의 correlation 정보를 더 많이 포함한다. 따라서, $x[n]$ 의 정보를 더 많이 반영할 수 있게 되고 이는 system을 더 정확히 추정할 수 있게 한다. 보라색 plot을 보게 되면, $|X(w)|^2$ 의 파형을 더 정확하게 표현하는 것을 확인할 수 있다.

하지만, pole의 개수가 많아진다고 system을 정확히 추정하는 것은 아니다. 실제 시스템의 pole 개수는 고정되어 있고, AR Modeling은 그걸 모르는 상태에서 추정하는 것이기 때문이다. Pole의 개수가 과도하게 많아지면 Overfitting등의 문제가 생기거나 너무 필요 이상으로 복잡한 model이 될 수 있다.

더불어, lag를 많이 사용하면 $r_{xx}[m]$ 를 더 많이 사용하기에, system의 과거 상태를 더 잘 반영한다. 하지만, Yule-Walker Equation에서 Matrix SIZE가 커지기에 계산량이 매우 많아지고, Correlation의 경우 lag가 커질수록 0에 가까워지기 때문에 ill-conditioned가 될 확률이 높아진다.

또한, 원본 신호 $x[n]$ 에 Noise가 섞여 있다면, pole의 개수가 많아지거나, lag가 많아졌을 때, Noise 까지도 추정할 수 있다. Noise의 경우에는 추정할 때 원하지 않는 신호이므로, 오히려 Model의 성능이 더 안 좋아질 수 있다.

Discussion



```
~~~~~
poles = roots(a_final);
disp("poles");
disp(poles);
~~~~~
```

```
poles
-0.7454 + 0.0000i
-0.6895 + 0.3687i
-0.6895 - 0.3687i
-0.4369 + 0.6206i
-0.4369 - 0.6206i
-0.1246 + 0.8730i
-0.1246 - 0.8730i
 0.4228 + 0.6897i
 0.4228 - 0.6897i
 0.9801 + 0.1416i
 0.9801 - 0.1416i
 0.8528 + 0.0000i
 0.7287 + 0.0000i
```

$A(z)$ 의 Coefficients들이 실수이므로 복소수 pole들은 항상 켤레 쌍으로 나오거나 실수로 나온다.

실수인 경우, pole은 real 축에 있으므로, $\omega = 0$ 또는 π 에 존재할 것임을 알 수 있다.

DSP 강의에서 pole의 위치에서는 peak를 가진다고 하였기에, 우리는 $\omega = 0$ 또는 π 에서 $1/A(\omega)$ 파형이 peak를 가진다고 추측할 수 있다.

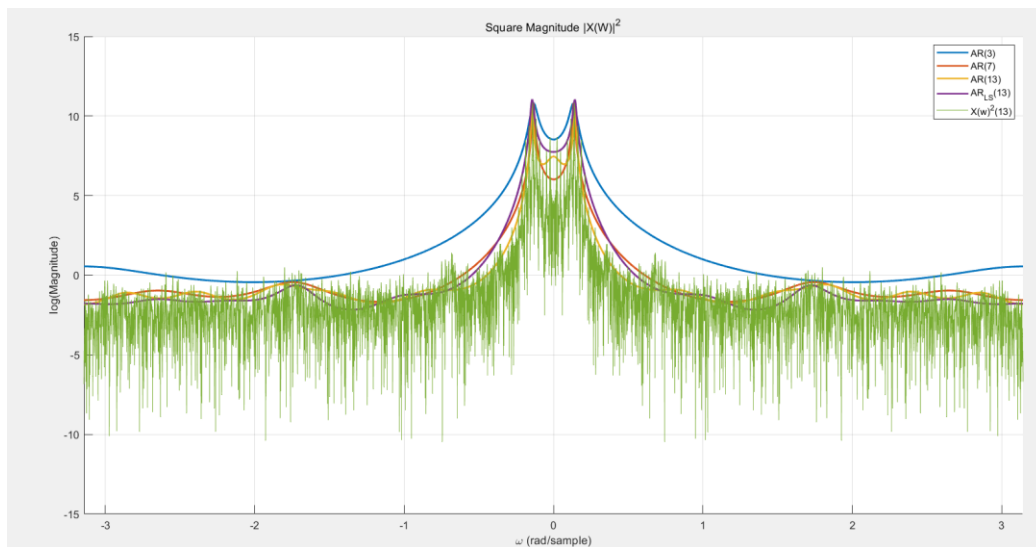
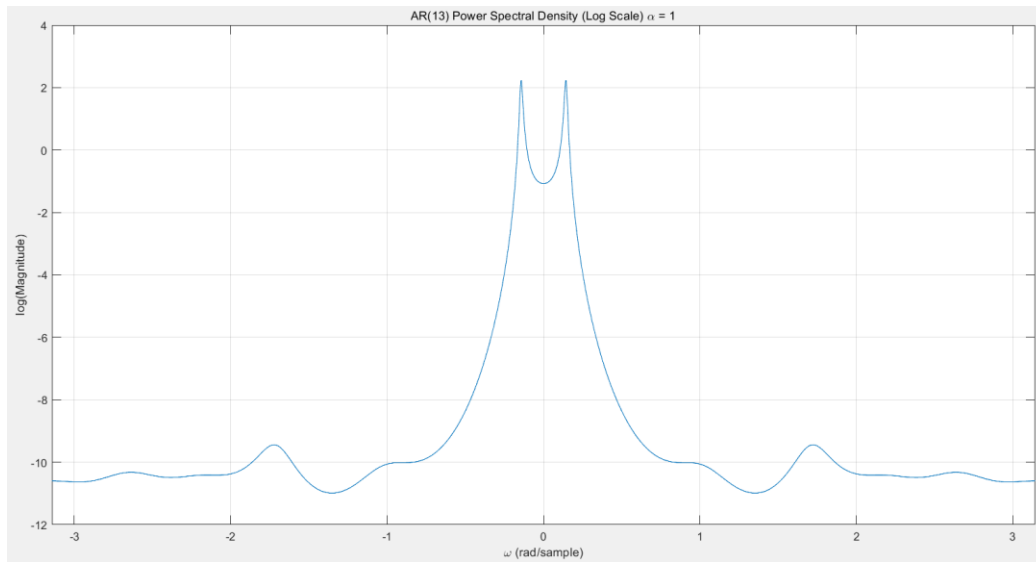
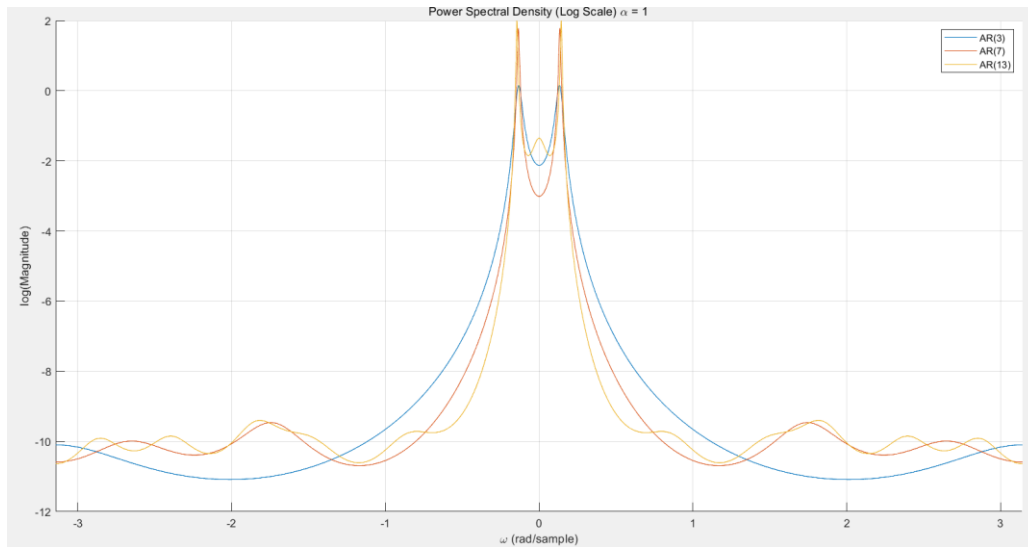
하지만, 위의 파형을 그려보면, $\omega = 0$ 또는 π 에서 peak가 나타나지 않고 있다.

이 이유를 살펴보면, 현재 plot은 FFT를 취하여 그리기 때문에 pole zero plot의 unit circle에서의 결과를 나타낸다고 볼 수 있을 것 같다.

하지만, 현재 실수 pole들의 값은 1이 아니고 unit circle에 가깝지 않은 위치에 존재한다고 볼 수 있다. 따라서, $\omega = 0$ 또는 π 에서 peak가 나타나지 않고 있다고 추측한다.

반면, $0.9801 \pm 0.1416i$, $0.4228 \pm 0.6897i$ 등의 복소수 pole을 보게 되면 실수인 pole에 비해 unit circle에 가까이 있기 때문에, 그 영향이 peak의 형태로 plot에 나타난다고 추측할 수 있다.

log10이 아닌 log로 plot 시 파형



Appendix

```
clear;      % 모든 변수 지우기
clc;        % 명령창 지우기
close all;  % 모든 figure 창 닫기

%% problem 3 & 4

% read x.wav & set FT of x
[x, fs] = audioread('x.wav');
x = x(:); % column vector
X_f = fftshift(fft(x, fs)); % shift to w=0, FFT
X_PowerSpec = abs(X_f).^2; % |X(w)|^2

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% beta = 1e-2;
beta_squared = 1e-4; % variance of signal
omega = linspace(-pi, pi, fs); % [-pi, pi]

% list of how many poles we are using, AR
p_list = [3, 7, 13];

figure;
hold on;
legend_names = {};

for idx = 1:length(p_list)
    p = p_list(idx);

    % compute r
    r0 = mean(x .* conj(x));
    r_tail = r_xx(x, p);
    r = [r0; r_tail];

    % Toeplitz Matrix
    R = toeplitz(r(1:p));
    r_vec = r(2:p+1);
    a_coeff = -inv(R) * r_vec;
    a_final = [1; a_coeff]; % note: a[0] = 1

    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % % Verify whether it is ok to us Toeplitz function
    % fprintf('\nr(1:%d) as row vector:\n\n', p);
    % row_r = r(1:p).'; % 열 벡터를 행 벡터로 변환 (transpose)
    %
    % for i = 1:p
    %     fprintf('%10.4e ', row_r(i)); % 지수 형식으로 출력
    % end
    % fprintf('\n');
    %
    % % Display header
    % fprintf('\nToeplitz Matrix R (%d x %d):\n\n', p, p);
    % fprintf('%6s', ''); % 빈 칸 (열 인덱스용)
    % for j = 1:p
    %     fprintf('%10d ', j); % 열 인덱스 출력
    % end
    % fprintf('\n');
    %
    % % Display matrix with row indices
    % for i = 1:p
    %     fprintf('%6d ', i); % 행 인덱스 출력
    %     for j = 1:p
    %         fprintf('%10.4e ', R(i,j)); % 지수형으로 값 출력
    %     end
    %     fprintf('\n');
    % end
    %%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
    % print
    fprintf('AR(%d) coefficients:\n', p);
    disp(a_final);

    % compute A(w)
    A_w = zeros(size(omega));
```

```

for k = 0:p
    A_w = A_w + a_final(k + 1) * exp(-1j * omega * k);
end

% compute PSD
PSD = beta_squared ./ abs(A_w).^2;

a = 1;
% a = max(X_PowerSpec) / max(PSD);

% Log scale
PSD_log = log10(a*PSD);
% PSD_log = log(a*PSD);
% PSD_log = log(1+PSD);

% Plot
plot(omega, PSD_log, 'DisplayName', sprintf('AR(%d)', p));
legend_names(end+1) = sprintf('AR(%d)', p);
end

title('Power Spectral Density (Log Scale) \alpha = 1');
xlabel('\omega (rad/sample)');
ylabel('log_1_0(Magnitude)');
legend show;
grid on;
xlim([-pi, pi]);

%% Problem 5

% settings
p = 13;          % number of poles
M = 2 * p;      % m = 1 ~ 2p
N = length(x);

% --- compute auto correlation r[1] ~ r[2p] ---
r_vec = r_xx(x, M); % Make Vector (SIZE: [2p x 1])

% --- R' Matrix (SIZE: [2p x p]) ---
R_prime = zeros(M, p);
r_full = [mean(x .* conj(x)); r_vec]; % r[0] is auto correlation with self

for m = 1:M
    for k = 1:p
        R_prime(m, k) = r_full(abs(m - k) + 1); % lag |m-k|
    end
end

% --- use Least Squares to compute a ---
a_coeff = -inv(R_prime' * R_prime) * R_prime' * r_vec;

% --- use a[0] = 1 to find final a_coefficients ---
a_final = [1; a_coeff];

% --- print ---
fprintf('AR(13) coefficients (Use Least Square solution):\n');
disp(a_final);

%% Problem 5
% --- compute PSD & log scale plot ---
beta_squared = 1e-4;
omega = linspace(-pi, pi, fs);
% a = 3000;
a = 1;

% A(w)
A_w = zeros(size(omega));
for k = 0:p
    A_w = A_w + a_final(k+1) * exp(-1j * omega * k);
end

% PSD
PSD = beta_squared ./ abs(A_w).^2;

% log scale
% PSD_log = log(a*PSD);
PSD_log = log10(a*PSD);

% --- plot ---
figure;
plot(omega, PSD_log);

```

```

title('AR(13) Power Spectral Density (Log Scale) \alpha = 1');
xlabel('\omega (rad/sample)');
ylabel('log_1_0(Magnitude)');
xlim([-pi, pi]);
grid on;

%% Problem 7

% --- 1. compute Power Spectrum |X(w)|^2 ---
X_f = fftshift(fft(x, fs)); % FT of x
X_PowerSpec = abs(X_f).^2; % |X(w)|^2

% log scale power spectrum
X_PowerSpec_log = log10(X_PowerSpec);
% X_PowerSpec_log = log(X_PowerSpec);

% n = 0:(length(x)-1);
% t = n / fs;
% disp(length(n));
% disp(length(x));
% figure;
% plot(t, x);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
figure;
hold on;
legend_names = {};
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
for idx = 1:length(p_list)
    p = p_list(idx);

    % compute r
    r0 = mean(x .* conj(x));
    r_tail = r_xx(x, p);
    r = [r0; r_tail];

    % Toeplitz Matrix
    R = toeplitz(r(1:p));
    r_vec = r(2:p+1);
    a_coeff = -inv(R) * r_vec;
    a_final = [1; a_coeff]; % note: a[0] = 1

    % print
    % fprintf('AR(%d) coefficients:\n', p);
    % disp(a_final);

    % compute A(w)
    A_w = zeros(size(omega));
    for k = 0:p
        A_w = A_w + a_final(k + 1) * exp(-1j * omega * k);
    end

    % compute PSD
    PSD = beta_squared ./ abs(A_w).^2;

    % a = 1;
    a = max(X_PowerSpec) / max(PSD);

    % Log scale
    PSD_log = log10(a*PSD);
    % PSD_log = log(a*PSD);
    % PSD_log = log(1+PSD);

    % Plot
    plot(omega, PSD_log, 'LineWidth', 1.5, 'DisplayName', sprintf('AR(%d)', p));
    legend_names{end+1} = sprintf('AR(%d)', p);
end

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%
omega = linspace(-pi, pi, fs);

% settings
p = 13; % number of poles
M = 2 * p; % m = 1 ~ 2p
N = length(x);

% --- compute auto crrelation r[1] ~ r[2p] ---
r_vec = r_xx(x, M); % Make Vector (SIZE: [2p x 1])

% --- R' Matrix (SIZE: [2p x p]) ---

```

```

R_prime = zeros(M, p);
r_full = [mean(x .* conj(x)); r_vec]; % r[0] is auto correlation with self

for m = 1:M
    for k = 1:p
        R_prime(m, k) = r_full(abs(m - k) + 1); % lag |m-k|
    end
end

% --- use Least Squares to compute a ---
a_coeff = -inv(R_prime' * R_prime) * R_prime' * r_vec;

% --- use a[0] = 1 to find final a_coefficients ---
a_final = [1; a_coeff];
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% --- compute PSD & log scale plot ---
beta_squared = 1e-4;
omega = linspace(-pi, pi, fs);
a = max(X_PowerSpec) / max(PSD);

% A(w)
A_w = zeros(size(omega));
for k = 0:p
    A_w = A_w + a_final(k+1) * exp(-1j * omega * k);
end

% PSD
PSD = beta_squared ./ abs(A_w).^2;

% log scale
% PSD_log = log(a*PSD);
PSD_log = log10(a*PSD);

% --- plot ---
plot(omega, PSD_log, 'LineWidth', 1.5, 'DisplayName', sprintf('AR_L_S(%d)', 13));
title('AR(13) Power Spectral Density (Log Scale)');
xlabel('\omega (rad/sample)');
ylabel('log_1_0(Magnitude)');
xlim([-pi, pi]);
grid on;

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% --- plot ---
plot(omega, X_PowerSpec_log, 'DisplayName', sprintf('X(w)^2(%d)', 13));
title('Square Magnitude |X(W)|^2');
xlabel('\omega (rad/sample)');
ylabel('log_1_0(Magnitude)');
legend;
grid on;
xlim([-pi, pi]);

%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

% poles = roots(a_final);
% disp("poles");
% disp(poles);
%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%%

figure;
subplot(2,1,1);
plot(omega, abs(log10(1./A_w)), 'LineWidth', 1.5);
title('1/A(w)');
xlabel('\omega (rad/sample)');
ylabel('log_1_0(Magnitude)');
xlim([-pi, pi]);
subplot(2,1,2);
plot(omega, angle(1./A_w), 'LineWidth', 1.5);
title('1/A(W) Phase');
xlabel('\omega (rad/sample)');
ylabel('angle(radian)');
xlim([-pi, pi]);

function output = r_xx(x,p)
    N = length(x);
    for i=1:p
        n=1:N-i;
        a=x(n+i).*conj(x(n));
    end
end

```



```
        output(i,1)=mean(a);  
    end  
end
```